

SCRIBE : Loop Engineering in Long Horizon Tasks Needs Agent to Write What They Did Each Turn (by Zane)

A new design paradigm and infra to LLM-agent training for Loop Engineering.

SCRIBE extends the classic ReAct loop with explicit `Reflect` and `Summarize` phases.

One trajectory is a loop, with many turns, and each turn with many steps. We don't need to worry about the number of steps will "burst" in a turn, because we have the sft data to warm up the model, and in rl phase its exploration space will not be too large to make the steps loop crazily in one turn.

Our infra now can collect the rollout data. In threshold mode the loop stops as soon as the submitted answer is correct, or alternatively when the turn reward reaches the pass threshold. Each turn has its own reward, and a trajectory reward is computed from all turns.

First : the context engineering of each turn:

Every LLM response follows this strict tag order. For convenience, we use T to stand for the think block, A to stand for the tool_call block, O to stand for the output block, R to stand for the reflect block, S to stand for the turn_summary block and AR to stand for the tool_response block. A turn's rollout usually is a `list[T/O/(list[A] + list[AR])] + R + S` structure. **(Note! in `list[T/O/(list[A] + list[AR])]` the actual rollout's input is always excluding T&R block because they are disposable! there we use `list[T/O/(list[A] + list[AR])]` because we show all the rollout blocks this turn generated and these infos are important when training.)** A step is one LLM call within a turn. A tool-call response triggers the next LLM call (i.e. the next step); the final non-tool LLM call ends the turn and produces `O + R + S`. So a turn with N tool calls has N + 1 steps. Each turn stores its input messages(`list[Dict]`), its new and rollout blocks(`list[ScribeBlocks]`) and its reward(int, the reward of a turn is equal to the reward of the last step in this turn. The reward of most steps in this turn are 0, the reward of a step is real only for the final non-tool step, because only that step can judge whether the submitted answer is correct.).

Block tags vs. message content

SCRIBE blocks are marked by explicit XML-style tags, **except** OUTPUT blocks which are bare text:

- `<think>...</think>` = THINK (T)
- `<tool_call>...</tool_call>` = TOOL_CALL (A)
- `<tool_response>...</tool_response>` = TOOL_RESPONSE (AR)
- `<reflect>...</reflect>` = REFLECT (R)
- `<turn_summary>...</turn_summary>` = TURN_SUMMARY (S)
- OUTPUT (O) has **no tags**.

A block and the OpenAI message that carries it are two different layers:

- **Block layer** (used for training/rollout): tags are always preserved so the parser can

split the stream into ScribeBlocks.

- **Message layer** (what the LLM actually reads in the next call):
 - `<think>` and `<reflect>` are **never** present in any future input; they are disposable the moment they are generated.
 - `<turn_summary>` is preserved with its tag in **recent** turns and also in **old** turns after compression. Keeping the tag makes block boundaries explicit.
 - `<tool_call>` is preserved with its tag whenever an old turn without S is reduced to O+A fallback.
 - OUTPUT is always bare text.
 - TOOL_CALL is carried by OpenAI's `tool_calls` field when the message also has tool calls; otherwise it may appear as a tagged block in `content`.
 - TOOL_RESPONSE is carried by `role="tool"` messages; the chat template renders them with `<tool_response>` tags, but the raw message content is bare.

History Compression rule:

1. Disposable T&R blocks: T and R blocks are disposable. Once generated, they are stripped before the next LLM call (next step) within the turn, and they never appear in future turns' inputs.
2. The turns before the recent k (default $k=3$) turns only preserve the S block (with its `<turn_summary>` tags intact).
3. A turn's rollout is atomic, so when the llm rollouts in a turn and suddenly the input tokens near the limit ($> \text{limit} - \text{small_number}$), compression will not be triggered temporarily. (this is a trade off for gambling the turn is not too long to make the llm's context crash, cause the loop engineering is a long horizon task but each turn is not so long, as each turn is improving a little in the former turns, maybe just the first turn is a big turn. However, the first turn's tokens use is hard to exceed the 1M context of sota llm nowadays, so in a word, the trade off for the atomic turn's rollout is reasonable. the output of a model has `max_output_limit`, if we set the `compression_limit - small_num` $600k (< < 1M)$, the system will be safe in most times.) The discipline is all about turn-level (not intra-turn level): The whole input blocks' tokens nears limit: Compression is triggered automatically **before rollout**. Two sets must be kept distinct:
 - **Token-count set (decides WHETHER to compress):** the **whole input context** — system prompt + first user prompt + every turn-feedback + all input blocks, counted together. The system prompt IS included in this count.
 - **Compression set (decides WHAT to compress):** grows by stage. The system prompt is NEVER compressed — it is always preserved verbatim (it carries the task/protocol and must stay identical across the trajectory). But the first user prompt and turn-feedbacks are NOT always preserved:
 - **Stage 1 (collect S blocks):** only the input blocks are touched; first user prompt + turn-feedbacks are preserved verbatim.
 - **Stage 2 (sota-LLM compression, triggered when the S-block sequence still nears the limit):** the LLM input INCLUDES the first user prompt and turn-feedbacks (see the explicit "the first user prompt and turn-feedbacks are included!" below) — they get folded into the single `<turn_summary>` output. Only the system prompt stays verbatim through both stages.

If the whole-input-context token count nears the limit, we compress **until it no longer nears the limit**, then proceed to rollout. This turn finally stores the compressed input and its new

and rollout blocks and its reward. The compression is firstly collecting all the S block in the input blocks as the compression result(new input blocks),the first user prompt and turn-feedback and the system prompt are ignored,then judge the length of the new input blocks(S block sequence),if the sequence's tokens nears limit again,use sota llm to do a compression(the first user prompt and turn-feedbacks are included!). The compressor returns JSON {"summary": "string"} and is validated to be exactly one `<turn_summary>...</turn_summary>` block; that block becomes the new compressed input. This one block's tokens should be guaranteed that they don't near the limit; if the sota llm output still exceeds the limit, retry the llm compression (the retry/backoff mechanism of the sota llm is decoupled from SCRIBE) until the single block fits.

Hard boundary: if the incompressible prefix — system prompt + first user prompt + all turn-feedbacks, with every input block already compressed away to nothing — still exceeds the limit on its own, compression cannot help. In that case SCRIBE raises an error (the trajectory config is infeasible: the preserved-verbatim parts alone overflow the context window).

if the turn's new block has no S block,that means this turn the llm's rollout format is wrong,when this turn need to only save the S block in context,just save O and A blocks instead (O as bare text, A with its `<tool_call>` tags).This is a special case,the below examples and statements all suppose the S block will be generated rightly in every turn.but if this special scene happens,you know you need to preserve O&A blocks instead when this turn is old enough to just need to preserve S block.And recent turns that do not have the S block just preserve O/A/AR blocks.

for instance:

In all examples below, S denotes the full `<turn_summary>...</turn_summary>` and A denotes the full `<tool_call>...</tool_call>` block (tags preserved

```
if k=3,max_turns=15
```

```
turn0:
system_prompt
first_user_prompt
list[T/O/(list[A]+list[AR])]+R+S,REWARD0
```

REWARD0 does not exceed the threshold.add feedback,
next turn:

```
turn1:
(input:
system_prompt
first_user_prompt
list[O/(list[A]+list[AR])]+S
feedback_from_turn0
)
system_prompt
first_user_prompt
list[O/(list[A]+list[AR])]+S
feedback_from_turn0
list[T/O/(list[A]+list[AR])]+R+S,
REWARD1
```

REWARD1 does not exceed the threshold.add feedback,
next turn:

```
turn2:
(input:
system_prompt
first_user_prompt
list[O/(list[A]+list[AR]))]+S
feedback_from_turn0
list[O/(list[A]+list[AR]))]+S
feedback_from_turn1
)
system_prompt
first_user_prompt
list[O/(list[A]+list[AR]))]+S
feedback_from_turn0
list[O/(list[A]+list[AR]))]+S
feedback_from_turn1
list[T/O/(list[A]+list[AR]))]+R+S,
REWARD2
```

REWARD2 does not exceed the threshold.add feedback,
next turn:

```
turn3:
system_prompt
first_user_prompt
list[O/(list[A]+list[AR]))]+S
feedback_from_turn0
list[O/(list[A]+list[AR]))]+S
feedback_from_turn1
list[O/(list[A]+list[AR]))]+S
feedback_from_turn2
list[T/O/(list[A]+list[AR]))]+R+S,
REWARD3
```

REWARD3 does not exceed the threshold.add feedback,
next turn:

```
turn4:
system_prompt
first_user_prompt
S
feedback_from_turn0
list[O/(list[A]+list[AR]))]+S
feedback_from_turn1
list[O/(list[A]+list[AR]))]+S
feedback_from_turn2
list[O/(list[A]+list[AR]))]+S
feedback_from_turn3
list[T/O/(list[A]+list[AR]))]+R+S,
REWARD4
```

REWARD4 does not exceed the threshold.add feedback,
next turn:

```
turn5:
system_prompt
first_user_prompt
S
feedback_from_turn0
S
feedback_from_turn1
list[O/(list[A]+list[AR]))]+S
feedback_from_turn2
list[O/(list[A]+list[AR]))]+S
feedback_from_turn3
list[O/(list[A]+list[AR]))]+S
feedback_from_turn4
list[T/O/(list[A]+list[AR]))]+R+S,
REWARD5
```

REWARD5 does not exceed the threshold.add feedback,
next turn:

```
turn6:
len(
system_prompt
first_user_prompt
S
feedback_from_turn0
S
feedback_from_turn1
list[O/(list[A]+list[AR]))]+S
feedback_from_turn2
list[O/(list[A]+list[AR]))]+S
feedback_from_turn3
list[O/(list[A]+list[AR]))]+S
feedback_from_turn4
list[O/(list[A]+list[AR]))]+S
feedback_from_turn5
) firstly > limit-small_number,compression triggered before rollout (ch
```

->

```
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
```

```
len(
system_prompt
first_user_prompt
```

```
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
) <= limit-small_number
```

```
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[T/O/(list[A]+list[AR])]+R+S,
REWARD6
```

REWARD6 does not exceed the threshold.add feedback,
next turn:

```
turn7 :
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])]+S
feedback_from_turn6
list[T/O/(list[A]+list[AR])]+R+S,
REWARD7
```

REWARD7 does not exceed the threshold.add feedback,
next turn:

```
turn8:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])]+S
feedback_from_turn6
list[O/(list[A]+list[AR])]+S
feedback_from_turn7
list[T/O/(list[A]+list[AR])]+R+S,
```

REWARD8

REWARD8 does not exceed the threshold.add feedback,
next turn:

turn9:

```
len(  
system_prompt  
first_user_prompt  
S feedback_from_turn0  
S feedback_from_turn1  
S feedback_from_turn2  
S feedback_from_turn3  
S feedback_from_turn4  
S feedback_from_turn5  
list[0/(list[A]+list[AR])] +S  
feedback_from_turn6  
list[0/(list[A]+list[AR])] +S  
feedback_from_turn7  
list[0/(list[A]+list[AR])] +S  
feedback_from_turn8  
)  
> limit-small_number,compression triggered.
```

->

```
system_prompt  
first_user_prompt  
S feedback_from_turn0  
S feedback_from_turn1  
S feedback_from_turn2  
S feedback_from_turn3  
S feedback_from_turn4  
S feedback_from_turn5  
S feedback_from_turn6  
S feedback_from_turn7  
S feedback_from_turn8
```

```
len(  
system_prompt  
first_user_prompt  
S feedback_from_turn0  
S feedback_from_turn1  
S feedback_from_turn2  
S feedback_from_turn3  
S feedback_from_turn4  
S feedback_from_turn5  
S feedback_from_turn6  
S feedback_from_turn7  
S feedback_from_turn8  
)<= limit-small_number
```

```
system_prompt  
first_user_prompt
```

```
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
S feedback_from_turn6
S feedback_from_turn7
S feedback_from_turn8
list[T/O/(list[A]+list[AR])]+R+S,
REWARD9
```

REWARD9 does not exceed the threshold.add feedback,
next turn:

```
turn10:
len(
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
S feedback_from_turn6
S feedback_from_turn7
S feedback_from_turn8
list[O/(list[A]+list[AR])]+S
feedback_from_turn9
)
>limit-small_number,compression triggered.
```

```
->
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
S feedback_from_turn6
S feedback_from_turn7
S feedback_from_turn8
S feedback_from_turn9
```

```
len(
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
```

```

S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
S feedback_from_turn6
S feedback_from_turn7
S feedback_from_turn8
S feedback_from_turn9
) still
> limit-small_number,compression triggered.

-> system_prompt S(call sota llm to summarize the input only except sys

system_prompt S
list[T/O/(list[A]+list[AR]])+R+S,REWARD10

turn11:

system_prompt S
list[O/(list[A]+list[AR]])+S
feedback_from_turn10
list[T/O/(list[A]+list[AR]])+R+S,REWARD11

The submitted answer is correct (or REWARD11 exceeds threshold)!!! In t

The trajectory finally is:

turn0:
input(please make the below system_prompt,user prompt and scribe blocks
system_prompt
first_user_prompt

new blocks(store the list[ScribeBlock]):
list[T/O/(list[A]+list[AR]])+R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A]])+R+S

reward(int):
REWARD0

---

turn1:
input(please make the below system_prompt,user prompt and scribe blocks
system_prompt
first_user_prompt
list[O/(list[A]+list[AR]])+S
feedback_from_turn0

new blocks(store the list[ScribeBlock]):
list[T/O/(list[A]+list[AR]])+R+S

rollout blocks(in order)(store the list[ScribeBlock]):

```

```
list[T/O/(list[A])]+R+S
```

```
reward(int):  
REWARD1
```

```
---
```

```
turn2:  
input:  
system_prompt  
first_user_prompt  
list[O/(list[A]+list[AR])]+S  
feedback_from_turn0  
list[O/(list[A]+list[AR])]+S  
feedback_from_turn1
```

```
new:  
list[T/O/(list[A]+list[AR])]+R+S
```

```
rollout blocks(in order)(store the list[ScribeBlock]):  
list[T/O/(list[A])]+R+S
```

```
reward(int):  
REWARD2
```

```
---
```

```
turn3:  
input:  
system_prompt  
first_user_prompt  
list[O/(list[A]+list[AR])]+S  
feedback_from_turn0  
list[O/(list[A]+list[AR])]+S  
feedback_from_turn1  
list[O/(list[A]+list[AR])]+S  
feedback_from_turn2
```

```
new:  
list[T/O/(list[A]+list[AR])]+R+S
```

```
rollout blocks(in order)(store the list[ScribeBlock]):  
list[T/O/(list[A])]+R+S
```

```
reward(int):  
REWARD3
```

```
---
```

```
turn4:  
input:  
system_prompt  
first_user_prompt
```

```

S
feedback_from_turn0
list[O/(list[A]+list[AR])] +S
feedback_from_turn1
list[O/(list[A]+list[AR])] +S
feedback_from_turn2
list[O/(list[A]+list[AR])] +S
feedback_from_turn3

new:
list[T/O/(list[A]+list[AR])] +R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])] +R+S

reward(int):
REWARD4

---

turn5:
input:
system_prompt
first_user_prompt
S
feedback_from_turn0
S
feedback_from_turn1
list[O/(list[A]+list[AR])] +S
feedback_from_turn2
list[O/(list[A]+list[AR])] +S
feedback_from_turn3
list[O/(list[A]+list[AR])] +S
feedback_from_turn4

new:
list[T/O/(list[A]+list[AR])] +R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])] +R+S

reward(int):
REWARD5

---

turn6:
input(Note!the input is the final compressed input):
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2

```

```
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
```

```
new:
```

```
list[T/O/(list[A]+list[AR])]+R+S
```

```
rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])]+R+S
```

```
reward(int):
REWARD6
```

```
---
```

```
turn7:
```

```
input:
```

```
system_prompt
```

```
first_user_prompt
```

```
S feedback_from_turn0
```

```
S feedback_from_turn1
```

```
S feedback_from_turn2
```

```
S feedback_from_turn3
```

```
S feedback_from_turn4
```

```
S feedback_from_turn5
```

```
list[O/(list[A]+list[AR])]+S
```

```
feedback_from_turn6
```

```
new:
```

```
list[T/O/(list[A]+list[AR])]+R+S
```

```
rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])]+R+S
```

```
reward(int):
REWARD7
```

```
---
```

```
turn8:
```

```
input:
```

```
system_prompt
```

```
first_user_prompt
```

```
S feedback_from_turn0
```

```
S feedback_from_turn1
```

```
S feedback_from_turn2
```

```
S feedback_from_turn3
```

```
S feedback_from_turn4
```

```
S feedback_from_turn5
```

```
list[O/(list[A]+list[AR])]+S
```

```
feedback_from_turn6
```

```
list[O/(list[A]+list[AR])]+S
```

```
feedback_from_turn7
```

```

new:
list[T/O/(list[A]+list[AR])]+R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])]+R+S

reward(int):
REWARD8

---

turn9:
input:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
S feedback_from_turn6
S feedback_from_turn7
S feedback_from_turn8

new:
list[T/O/(list[A]+list[AR])]+R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])]+R+S

reward(int):
REWARD9

---

turn10:
input:
system_prompt S

new:
list[T/O/(list[A]+list[AR])]+R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])]+R+S

reward(int):
REWARD10

---

turn11:

```

```

input :
system_prompt S
list[O/(list[A]+list[AR]))]+S
feedback_from_turn10

new:
list[T/O/(list[A]+list[AR]))]+R+S

rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A]))]+R+S

reward(int):
REWARD11

```

Second : the reward rule of a turn is:

the metrics(MULTI-DIMENSIONAL REWARD PREVENTS HACKING) (**ALL THE METRICS ARE JUST FOCUED ON THIS TURN'S NEW BLOCKS,NOT THE INPUT BLOCKS**) :

1. the reward of the final step. It is the product of three factors: (a) **answer correctness** — 1.0 if the submitted answer matches the right answer, else 0.0; (b) **natural termination bonus** — the turn must end with a final non-tool LLM call (i.e. `terminated=True` and `done_reason=="no_tool_call"`). If the turn ends by **hitting** `max_steps` (`truncated=True`) or by a tool-call step, this factor is 0.0. Block-level format (O/R/S presence/order) is intentionally handled by metric 4, not here, so metric 1 stays orthogonal for ablations; (c) **truncated penalty** — if the turn is truncated by the step limit before natural termination, apply an additional penalty (e.g. -0.2). In short, only a correct answer that is produced by a clean final non-tool step can receive the full final-step reward.
2. the tool usage quality in the rollout blocks of this turn: (a) **submit count** — submit must be called exactly once per turn; (b) **parallel identical tool calls in a single LLM call are penalized** — if an assistant message contains multiple tool_calls with the **same function name AND the same arguments**, each duplicated call incurs a penalty (e.g. -0.2 per duplicated call). "Same" means an exact match on both `name` and `normalized arguments` (e.g. `json.dumps(args, sort_keys=True)`). Cross-step repetitions are NOT penalized, because the environment state may have changed between steps (e.g. `run` → `edit` → `run` the same command again).
3. the length of the steps used.(computed by the number of A,keep the metric 1 the same level while make metric 3 as lower as possible)
4. the format correctness. This metric checks: (a) the turn contains the expected block types: `think`, `output`, `tool_call`, `tool_response`, `reflect`, `turn_summary`; (b) in the final non-tool step, the message must end with **exactly one block followed by exactly one** `<turn_summary>` **block** (reflect is the second-to-last block, `turn_summary` is the last block). Missing either block, or having them in the wrong order, is a format error; (c) **and** `<turn_summary>` **appear only in the final step** — any R/S block in a non-final step is a format error; (d) **the final non-tool step must contain NO tool_calls**: an assistant message that mixes `<turn_summary>` with `tool_call(s)` is invalid, because `R + S` must be plain text; (e) **only the agreed-upon SCRIBE tags are allowed** — any tag other than `<think>`, `<tool_call>`, `<tool_response>`, `<reflect>`, and `<turn_summary>` is invalid. Examples of invalid tags include `<submit>`, `<bash>`,

<action>, <plan>, etc.; (f) **R + S may only appear after submit has been called in this turn** — a turn that ends with R + S but never called submit, or where R/S appears in the same step as or before the submit call, is invalid; (g) how much of the turn's parsed data is invalid (see the code's invalid detection).

5. the total tokens rollouted out in this turn(namely rollout blocks,include think,output,tool_call,reflect and turn_summary block,exclude the tool_response block.This metric is larger,the reward is lower,but should not effect reward so much)
6. token reuse ratio, $\text{reuse} = |S \cap (\text{OUA})| / |\text{OUA}|$ where S = token set of the turn_summary block, O = token set of the output block, A = token set of the tool_call block. The denominator is OUA only (S is excluded from the denominator on purpose, so the ratio measures "what fraction of the ground-truth key tokens the summary reuses" and is not inflated by the summary's own length).
7. the faithfulness of the turn summary— did the summary tell the truth?(use llm as a judge to give a score of the faithfulness of the turn summary,the turn summary should be a statement of what we do in the output and tool_call block.The judge compares the summary against the ground-truth blocks(output and tool_call blocks) and scores how consistent they are.) High score is the summary accurately reports what was thought and done,while the low score is the summary fabricates, omits, or contradicts the real behavior.
8. the direction neutrality - did the summary stay retrospective?The turn summary must only **look back** at the current turn. It must not predict, plan, or prescribe what the *next* turn should do.High score is the summary purely recaps,while the low score is the summary leaks into future planning.use llm as a judge.
9. the turn focus of the turn summary - does the summary describe only this turn? Then turn summary must be scoped strictly to the **current** turn.It must not mix in content from **earlier** turns - each turn's summary stands alone as the record of that turn alone. (Note: later turns' content cannot appear by construction — the summary is generated at the end of the current turn, before any subsequent turn exists. So this dimension only guards against bleeding **prior** turns in.) High score is the summary describes only what happened in this turn,while the low score is the summary folds prior turns' actions into the current one. use llm as a judge.
10. the fluency of the turn summary,use llm as a judge.we need to improve the token reuse ratio metric,but if we pile up the key tokens awkwardly,that sucks.so we need to judge the fluency of the turn summary to make the turn summary fluent and natural.
11. the compression ratio of the summary,to improve the the token reuse ratio metric and the fluency of the turn summary,the model maybe use the whole ground-truth blocks(output and tool_call blocks) as a turn summary,that sucks.so we need to count the compression ratio of the summary: $\text{len}(\text{the token set in turn_summary block}) / \text{len}(\text{the token set in output,tool_call block})$,this metric lower,the reward higher.This is a pure statistical metric (no LLM judge needed).
12. Cross block N-gram overlap penalty,penalize copy-paste from ground-truth blocks.Even with #11 (compression ratio) blocking wholesale copying, the model may **partially** copy-paste: lifting consecutive phrases verbatim from the ground-truth blocks into the summary. This slips past #11 (not long enough to tank compression) and may even pass #10 (fluent, since it's real text),yet it is not genuine summarization. This metric computes the overlap of **contiguous n-grams** ($n = 4$ recommended) between the

turn_summary block and the ground-truth (output + tool_call) blocks: $\text{shared} = \text{set}(\text{ngrams}(\text{summary}, 4)) \cap \text{set}(\text{ngrams}(\text{ground_truth}, 4))$ $\text{overlap_ratio} = |\text{shared}| / |\text{set}(\text{ngrams}(\text{summary}, 4))|$ Higher overlap $\rightarrow$ higher penalty $\rightarrow$ lower reward. It is a pure statistical metric (no LLM judge needed), cheap to compute. low overlap is the summary rephrases in its own words, the high overlap is the summary lifts verbatim runs from ground-truth.

13. Intra-block N-gram Overlap Penalty — penalize repetition within a block. #12 only catches the summary copying from ground-truth (cross-block). It cannot see a block repeating *itself internally* — a common small-model /early-RL failure where a block loops the same phrase to pad length. For each model-generated block (think / output / reflect / turn_summary), compute the share of its n-grams ($n=4$) that are redundant due to internal repetition:

```
def intra_overlap(block_tokens, n=4):
    ngrams = [tuple(block_tokens[i:i+n])
               for i in range(len(block_tokens) - n + 1)]
    total = len(ngrams)
    unique = len(set(ngrams))
    return (total - unique) / total if total else 0.0
```

Block-level ratios are aggregated (e.g. weighted mean) into one penalty. Pure statistics, no judge. Scope: only model-generated blocks — tool_call (structured JSON, naturally repetitive) and tool_response(env-produced, not the model's responsibility) are excluded. the metric is low, that means the block does not repeat itself, otherwise the block loops.

14. **Malformed tool-call penalty** — penalize tool calls whose arguments fail schema-level validation or cause execution errors due to bad arguments. A call is malformed if the execution layer reports `error_type == "malformed_arguments"` or `error_type == "exec_error"`: for example, the argument JSON is unparseable, arguments is not an object, a required parameter (such as bash's command) is missing, or an unknown/extra parameter (such as bash's hallucinated stdout) is passed. Each malformed call incurs a fixed penalty (e.g. -0.1) and the metric is clamped to [0,1]. Perfect tool usage $\rightarrow$ 1.0. This gives GRPO a continuous gradient to suppress malformed calls instead of letting them loop until truncation.
15. **Repeated tool-call penalty** — penalize the model emitting the exact same (name, normalized arguments) tool call that was already executed earlier in this turn. Unlike metric 2b (which only penalizes parallel duplicates within one assistant message), this metric catches cross-step loops such as `bash {command: ls} -> result -> bash {command: ls}` again. Each repeated call incurs a fixed penalty (e.g. -0.08) and the metric is clamped to [0,1]. Perfect usage $\rightarrow$ 1.0. Together with metric 14, this stops the common rollout failure mode where the model repeats or malforms tool calls and wastes steps until `max_steps_per_turn` truncates the turn.

Note : the 7,8,9,10 metric will be given completely in one llm call. (Metric 11 is a pure statistical metric and stays out of this judge call — see its line "no LLM judge needed".)

Note : the reflect block is a cot process for model to let the latter turn_summary get higher reward.

Third : the reward rule of a trajectory is:

`trajectory_reward = mean(turn_reward for turn in turns_used) * decay ** len(turns_used)`

where `turn_reward` is the turn-level reward of each turn (0~1, from the 15 metrics), `len(turns_used)` is the number of turns actually used in this trajectory, and `decay` is a discount factor in (0,1) (e.g. 0.8).

We use the **mean** turn reward rather than only the final turn reward so that poor behavior in earlier turns is directly penalized. A trajectory that wastes early turns with malformed or repeated tool calls receives a lower trajectory reward even if the final turn eventually succeeds. The length decay still rewards solving in fewer turns: a 1-turn success gives $\text{mean_reward} * \text{decay}$, while a 3-turn success gives $\text{mean_reward} * \text{decay}^3$.

Fourth : RL Training and Credit Assignment:

we use grpo for different trajectory.

- not reinforce: not stable, variance so large
- not ppo: critic model is expensive
- but grpo: stable enough, more effective

The reward→credit pipeline (written out explicitly to avoid confusing "reward" with "credit"):

1. **trajectory_reward** (from Section Third) is the quantity GRPO compares across the group of trajectories sampled on the same task. The GRPO group baseline = mean of trajectory_reward over the group; each trajectory's **advantage** = (its trajectory_reward - group mean) / group std.
2. **turn-level credit**: the trajectory advantage is shared equally across all turns in that trajectory → each turn's credit = trajectory_advantage / len(turns_used).

A turn may have the data structure: `input(list[Dict]) new(list[ScribeBlock])`
`rollout(list[ScribeBlock])`

we need to use this structure to convert to:

`list[Step]`

Step: `input:list[Dict] output:list[ScribeBlock]`

In the step-level examples below, `O A AR` is an abstract block-level shorthand: the actual input is a list of OpenAI messages (assistant message with `OUTPUT` + `tool_calls` for A, followed by tool-role messages for AR). T/R are stripped from the assistant message content before appending, but A is carried by the `tool_calls` field, not by tags in content.

for instance:

we have turn7:

```
turn7:
input:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
```

```
S feedback_from_turn5
list[O/(list[A]+list[AR])] +S
feedback_from_turn6
```

```
new:
list[T/O/(list[A]+list[AR])] +R+S
```

```
rollout blocks(in order)(store the list[ScribeBlock]):
list[T/O/(list[A])] +R+S
```

```
reward(int):
REWARD7
```

we have steps of turn7:

step0:

```
input:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])] +S
feedback_from_turn6
```

```
output:
T O A
```

step1:

```
input:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])] +S
feedback_from_turn6
O A AR
```

```
output:
O A
```

step2:

```
input:
```

```

system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])] + S
feedback_from_turn6
O A AR
O A AR

```

```

output:
O T O A

```

```

step3 :

```

```

input:
system_prompt
first_user_prompt
S feedback_from_turn0
S feedback_from_turn1
S feedback_from_turn2
S feedback_from_turn3
S feedback_from_turn4
S feedback_from_turn5
list[O/(list[A]+list[AR])] + S
feedback_from_turn6
O A AR
O A AR
O O A AR

```

```

output:
O R S

```

So in a word, we can get `list[Step]` from a turn.

3. **Step-level credit:** the turn's credit is shared equally across all steps in that turn $\rightarrow$ each step's credit = $\text{turn_credit} / \text{num_steps_in_turn}$.
4. **token-level credit:** the step's credit is shared equally across all rollout tokens in that step $\rightarrow$ each rollout token's credit = $\text{step_credit} / \text{num_rollout_tokens_in_step}$. (Rollout tokens = model-generated tokens only: think / output / tool_call / reflect / turn_summary blocks. (Step.output) Input blocks, tool_response, system/kickoff/feedback are masked — not model-generated.)

So the chain is fixed: **trajectory_reward** $\rightarrow$ (GRPO group) $\rightarrow$ **trajectory_advantage** $\rightarrow$ ($\div$ turns) $\rightarrow$ **turn credit** $\rightarrow$ ($\div$ steps) $\rightarrow$ **step credit** $\rightarrow$ ($\div$ rollout tokens) $\rightarrow$ **token credit**, and the token credit is what multiplies $\nabla \log P(\text{token})$ in the policy gradient.

we have TOKEN-LEVEL CREDIT ASSIGNMENT, because the turn_summary has the token reuse ratio.

Operations Notes

vLLM runtime LoRA adapter loading

If online GRPO crashes at `update_weights` with a 404 for `/v1/load_lora_adapter`:

1. vLLM only registers the admin endpoint when the environment variable is set:

```
VLLM_ALLOW_RUNTIME_LORA_UPDATING=1 vllm serve ... --enable-lora
```

2. The `VLLMBackend.base_url` already ends in `/v1`, so the backend must call `/load_lora_adapter` (not `/v1/load_lora_adapter`) to form the correct URL `http://localhost:8000/v1/load_lora_adapter`.

Symptom before fix: 404 Not Found for `.../v1/load_lora_adapter` (missing env) or `.../v1/v1/load_lora_adapter` (double `/v1`).

Submit-only ablation data quality gate

For the submit-only SCRIBE ablation, the SFT collection script filters turns on the fly: only turns with `turn_reward >= 0.7` and `metric_4 (format) >= 0.8` are kept. This removes the common failure mode where the model writes literal SCRIBE tag names such as `<reflect>` inside a `<think>` block, which the parser treats as nested tags and rejects.